Behavioural Modelling & Timing in Verilog

Behavioral models in Verilog contain procedural statements, which control the simulation and manipulate variables of the data types. These all statements are contained within the procedures. Each of the procedure has an activity flow associated with it.

During simulation of behavioral model, all the flows defined by the always and initial statements start together at simulation time zero. The initial statements are executed once, and the always statements are executed repetitively. In this model, the register variables a and b are initialized to binary 1 and 0 respectively at simulation time zero. The initial statement is then completed and is not executed again during that simulation run. This initial statement is containing a begin-end block (also called a sequential block) of statements. In this begin-end type block, a is initialized first followed by b.

Example of Behavioral Modeling

```
module behave;
reg [1:0]a,b;

initial
begin
   a = b1;
   b = b0;
end

always
begin
   #50 a = ~a;
end

always
begin
   #100 b = ~b;
```

```
end
End module
```

Procedural Assignments

Procedural assignments are for updating reg, integer, time, and memory variables. There is a significant difference between procedural assignment and continuous assignment as described below −

Continuous assignments drive net variables and are evaluated and updated whenever an input operand changes value.

Procedural assignments update the value of register variables under the control of the procedural flow constructs that surround them.

The right-hand side of a procedural assignment can be any expression that evaluates to a value. However, part-selects on the right-hand side must have constant indices. The lefthand side indicates the variable that receives the assignment from the right-hand side. The left-hand side of a procedural assignment can take one of the following forms −

- register, integer, real, or time variable − An assignment to the name reference of one of these data types.
- bit-select of a register, integer, real, or time variable − An assignment to a single bit that leaves the other bits untouched.
- part-select of a register, integer, real, or time variable − A part-select of two or more contiguous bits that leaves the rest of the bits untouched. For the part-select form, only constant expressions are legal.
- memory element − A single word of a memory. Note that bit-selects and part-selects are illegal on memory element references.
- concatenation of any of the above − A concatenation of any of the previous four forms can be specified, which effectively partitions the result of the right-hand side expression and assigns the partition parts, in order, to the various parts of the concatenation.

Delay in Assignment (not for synthesis)

In a delayed assignment t time units pass before the statement is executed and the lefthand assignment is made. With intra-assignment delay, the right side is evaluated immediately but there is a delay of t before the result is place in the left hand assignment. If another procedure changes a right-hand side signal during t, it does not effect the output. Delays are not supported by synthesis tools.

Syntax

- **Procedural Assignment** variable = expression
- **Delayed assignment** #t variable = expression;

- **Intra-assignment delay** variable = #t expression;

Example

```
reg [6:0] sum; reg h, ziltch;
sum[7] = b[7] ^ c[7]; // execute now.
ziltch = #15 ckz&h; /* ckz&a evaluated now; ziltch changed
after 15 time units. */

#10 hat = b&c; /* 10 units after ziltch changes, b&c is
evaluated and hat changes. */
```

Blocking Assignments

A blocking procedural assignment statement must be executed before the execution of the statements that follow it in a sequential block. A blocking procedural assignment statement does not prevent the execution of statements that follow it in a parallel block.

Syntax

The syntax for a blocking procedural assignment is as follows –

```
<lvalue> = <timing_control> <expression>
```

Where, lvalue is a data type that is valid for a procedural assignment statement, = is the assignment operator, and timing control is the optional intra - assignment delay. The timing control delay can be either a delay control (for example, #6) or an event control (for example, @(posedge clk)). The expression is the right-hand side value the simulator assigns to the left-hand side. The = assignment operator used by blocking procedural assignments is also used by procedural continuous assignments and continuous assignments.

Example

```
rega = 0;
rega[3] = 1;            // a bit-select
rega[3:5] = 7;          // a part-select
mema[address] = 8hff;  // assignment to a memory element
{carry, acc} = rega &plus; regb;  // a concatenation
```

Nonblocking (RTL) Assignments

The non-blocking procedural assignment allows you to schedule assignments without blocking the procedural flow. You can use the non-blocking procedural statement whenever you want to make

several register assignments within the same time step without regard to order or dependance upon each other.

Syntax

The syntax for a non-blocking procedural assignment is as follows –

```
<lvalue> <= <timing_control> <expression>
```

Where lvalue is a data type that is valid for a procedural assignment statement, <= is the non-blocking assignment operator, and timing control is the optional intra-assignment timing control. The timing control delay can be either a delay control or an event control (for example, @(posedge clk)). The expression is the right-hand side value the simulator assigns to the left-hand side. The non-blocking assignment operator is the same operator the simulator uses for the less-than-orequal relational operator. The simulator interprets the <= operator to be a relational operator when you use it in an expression, and interprets the <= operator to be an assignment operator when you use it in a non-blocking procedural assignment construct.

How the simulator evaluates non-blocking procedural assignments When the simulator encounters a non-blocking procedural assignment, the simulator evaluates and executes the non-blocking procedural assignment in two steps as follows –

- The simulator evaluates the right-hand side and schedules the assignment of the new value to take place at a time specified by a procedural timing control. The simulator evaluates the right-hand side and schedules the assignment of the new value to take place at a time specified by a procedural timing control.
- At the end of the time step, in which the given delay has expired or the appropriate event has taken place, the simulator executes the assignment by assigning the value to the left-hand side.

Example

```
module evaluates2(out);
output out;
reg a, b, c;
initial

begin
   a = 0;
   b = 1;
   c = 0;
end
always c = #5 ~c;
always @(posedge c)

begin
```

```
   a <= b;
   b <= a;
end
endmodule
```

Conditions

The conditional statement (or if-else statement) is used to make a decision as to whether a statement is executed or not.

Formally, the syntax is as follows –

```
<statement>
::= if ( <expression> ) <statement_or_null>
||= if ( <expression> ) <statement_or_null>
   else <statement_or_null>
<statement_or_null>

::= <statement>
||= ;
```

The <expression> is evaluated; if it is true (that is, has a non-zero known value), the first statement executes. If it is false (has a zero value or the value is x or z), the first statement does not execute. If there is an else statement and <expression> is false, the else statement executes. Since, the numeric value of the if expression is tested for being zero, certain shortcuts are possible.

For example, the following two statements express the same logic –

```
if (expression)
if (expression != 0)
```

Since, the else part of an if-else is optional, there can be confusion when an else is omitted from a nested if sequence. This is resolved by always associating the else with the closest previous if that lacks an else.

Example

```
if (index > 0)
if (rega > regb)
   result = rega;
   else // else applies to preceding if
   result = regb;

If that association is not what you want, use a begin-end block statement
```

```
to force the proper association

if (index > 0)
begin

if (rega > regb)
result = rega;
end
   else
   result = regb;
```

Construction of: if- else- if

The following construction occurs so often that it is worth a brief separate discussion.

Example

```
if (<expression>)
   <statement>
   else if (<expression>)
   <statement>
   else if (<expression>)
   <statement>
   else
   <statement>
```

This sequence of ifs (known as an if-else-if construct) is the most general way of writing a multi-way decision. The expressions are evaluated in order; if any expression is true, the statement associated with it is executed, and this terminates the whole chain. Each statement is either a single statement or a block of statements.

The last else part of the if-else-if construct handles the none of the above or default case where none of the other conditions was satisfied. Sometimes there is no explicit action for the default; in that case, the trailing else can be omitted or it can be used for error checking to catch an impossible condition.

Case Statement

The case statement is a special multi-way decision statement that tests whether an expression matches one of a number of other expressions, and branches accordingly. The case statement is useful for describing, for example, the decoding of a microprocessor instruction. The case statement has the following syntax –

Example

```
<statement>
::= case ( <expression> ) <case_item>&plus; endcase
```

```
||= casez ( <expression> ) <case_item>&plus; endcase
||= casex ( <expression> ) <case_item>&plus; endcase
<case_item>
::= <expression> <,<expression>>* : <statement_or_null>
||= default : <statement_or_null>
||= default <statement_or_null>
```

The case expressions are evaluated and compared in the exact order in which they are given. During the linear search, if one of the case item expressions matches the expression in parentheses, then the statement associated with that case item is executed. If all comparisons fail, and the default item is given, then the default item statement is executed. If the default statement is not given, and all of the comparisons fail, then none of the case item statements is executed.

Apart from syntax, the case statement differs from the multi-way if-else-if construct in two important ways –

- The conditional expressions in the if-else-if construct are more general than comparing one expression with several others, as in the case statement.
- The case statement provides a definitive result when there are x and z values in an expression.

Looping Statements

There are four types of looping statements. They provide a means of controlling the execution of a statement zero, one, or more times.

- forever continuously executes a statement.
- repeat executes a statement a fixed number of times.
- while executes a statement until an expression becomes false. If the expression starts out false, the statement is not executed at all.
- for controls execution of its associated statement(s) by a three-step process, as follows –
  - Executes an assignment normally used to initialize a variable that controls the number of loops executed
  - Evaluates an expressionif the result is zero, the for loop exits, and if it is not zero, the for loop executes its associated statement(s) and then performs step 3
  - Executes an assignment normally used to modify the value of the loopcontrol variable, then repeats step 2

The following are the syntax rules for the looping statements –

Example

```
<statement>
::= forever <statement>
||=forever
begin
   <statement>&plus;
end

<Statement>
::= repeat ( <expression> ) <statement>
||=repeat ( <expression> )
begin
   <statement>&plus;
end

<statement>
::= while ( <expression> ) <statement>
||=while ( <expression> )
begin
   <statement>&plus;
end
<statement>
::= for ( <assignment> ; <expression> ; <assignment> )
<statement>
||=for ( <assignment> ; <expression> ; <assignment> )
begin
   <statement>&plus;
end
```

Delay Controls

Delay Control

The execution of a procedural statement can be delay-controlled by using the following syntax –

```
<statement>
::= <delay_control> <statement_or_null>
<delay_control>
::= # <NUMBER>
||= # <identifier>
||= # ( <mintypmax_expression> )
```

The following example delays the execution of the assignment by 10 time units –

```
#10 rega = regb;
```

The next three examples provide an expression following the number sign (#). Execution of the assignment delays by the amount of simulation time specified by the value of the expression.

Event Control

The execution of a procedural statement can be synchronized with a value change on a net or register, or the occurrence of a declared event, by using the following event control syntax –

Example

```
<statement>
::= <event_control> <statement_or_null>

<event_control>
::= @ <identifier>
||= @ ( <event_expression> )

<event_expression>
::= <expression>
||= posedge <SCALAR_EVENT_EXPRESSION>
||= negedge <SCALAR_EVENT_EXPRESSION>
||= <event_expression> <or <event_expression>>
```

*<SCALAR_EVENT_EXPRESSION> is an expression that resolves to a one bit value.

Value changes on nets and registers can be used as events to trigger the execution of a statement. This is known as detecting an implicit event. Verilog syntax also allows you to detect change based on the direction of the changethat is, toward the value 1 (posedge) or toward the value 0 (negedge). The behaviour of posedge and negedge for unknown expression values is as follows –

- a negedge is detected on the transition from 1 to unknown and from unknown to 0
- a posedge is detected on the transition from 0 to unknown and from unknown to 1

Procedures: Always and Initial Blocks

All procedures in Verilog are specified within one of the following four Blocks. 1) Initial blocks 2) Always blocks 3) Task 4) Function

The initial and always statements are enabled at the beginning of simulation. The initial blocks executes only once and its activity dies when the statement has finished. In contrast, the always blocks executes repeatedly. Its activity dies only when the simulation is terminated. There is no limit to the number of initial and always blocks that can be defined in a module. Tasks and functions are procedures that are enabled from one or more places in other procedures.

Initial Blocks

The syntax for the initial statement is as follows –

```
<initial_statement>
::= initial <statement>
```

The following example illustrates the use of the initial statement for initialization of variables at the start of simulation.

```
Initial
Begin
   Areg = 0; // initialize a register
   For (index = 0; index < size; index = index &plus; 1)
   Memory [index] = 0; //initialize a memory
   Word
End
```

Another typical usage of the initial Blocks is specification of waveform descriptions that execute once to provide stimulus to the main part of the circuit being simulated.

```
Initial
Begin
   Inputs = b000000;
   // initialize at time zero
   #10 inputs = b011001; // first pattern
   #10 inputs = b011011; // second pattern
   #10 inputs = b011000; // third pattern
   #10 inputs = b001000; // last pattern
End
```

Always Blocks

The always statement repeats continuously throughout the whole simulation run. The syntax for the always statement is given below

```
<always_statement>
::= always <statement>
```

The always statement, because of its looping nature, is only useful when used in conjunction with some form of timing control. If an always statement provides no means for time to advance, the always statement creates a simulation deadlock condition. The following code, for example, creates an infinite zero-delay loop –

```
Always areg = ~areg;
```

Providing a timing control to the above code creates a potentially useful descriptionas in the following example –

```
Always #half_period areg = ~areg;
```


TOP TUTORIALS

Python Tutorial
Java Tutorial
C++ Tutorial
C Programming Tutorial
C# Tutorial
PHP Tutorial
R Tutorial
HTML Tutorial
CSS Tutorial
JavaScript Tutorial
SQL Tutorial

TRENDING TECHNOLOGIES

Cloud Computing Tutorial
Amazon Web Services Tutorial
Microsoft Azure Tutorial
Git Tutorial
Ethical Hacking Tutorial
Docker Tutorial
Kubernetes Tutorial
DSA Tutorial
Spring Boot Tutorial
SDLC Tutorial
Unix Tutorial

CERTIFICATIONS

Business Analytics Certification
Java & Spring Boot Advanced Certification
Data Science Advanced Certification
Cloud Computing And DevOps
Advanced Certification In Business Analytics
Artificial Intelligence And Machine Learning
DevOps Certification
Game Development Certification
Front-End Developer Certification
AWS Certification Training
Python Programming Certification

COMPILERS & EDITORS

Online Java Compiler
Online Python Compiler
Online Go Compiler
Online C Compiler
Online C++ Compiler
Online C# Compiler
Online PHP Compiler
Online MATLAB Compiler
Online Bash Terminal
Online SQL Compiler
Online Html Editor

ABOUT US | OUR TEAM | CAREERS | JOBS | CONTACT US | TERMS OF USE |
PRIVACY POLICY | REFUND POLICY | COOKIES POLICY | FAQ'S

tutorialspoint

GET IT ON Google Play

Download on the App Store

Tutorials Point is a leading Ed Tech company striving to provide the best learning material on technical and non-technical subjects.

© Copyright 2026. All Rights Reserved.